

auto_ml

이진 분류(0/1) 문제를 자동으로 학습·스코어링하는 사내 라이브러리. 폐쇄 환경 이관 / 운영을 전제로 모듈화·설정 주도(YAML)로 설계되었다.

구성

auto_ml/	
├── config.py	설정 dataclass + YAML 로더
├── pipeline.py	학습 파이프라인 (auto-ml-train)
├── preprocessing/	1) 결측 → 2) 이상치 → 2.5) skew 변환 → 3) 스케일링
├── feature_selection/	Stability Selection 변수 선택
├── models/	LGBM / XGBoost / CatBoost 래퍼 + Trainer
├── tuning/	Optuna 베이지안 하이퍼파라미터 최적화
├── reporting/	HTML + PDF 리포트 (동일 내용)
├── scoring/	배치 스코어링 (auto-ml-score)
└── utils/	io / logger / validation

5단계 파이프라인

1. **전처리** — 결측 → 이상치 → skew 변환 → 스케일링 순서 고정. 학습 시 통계량을 저장해 스코어링 시 동일 적용.
2. **변수 선택** (선택) — Stability Selection (Meinshausen & Bühlmann, 2010). 전처리 직후 적용하여 안정적으로 선택되는 변수만 모델에 전달한다. `feature_selection.enabled: false` (기본값) 이면 건너뛴다.
3. **모형 적합** — LGBM / XGBoost / CatBoost 3종에 대해 (a) Optuna TPE 로 하이퍼파라미터 베이지안 최적화 → (b) StratifiedKFold OOF 평가 → (c) 학습 전체로 최종 fit → (d) 테스트 데이터로 평가. `primary_metric` (기본 ROC-AUC) 기준으로 best 모델을 선정한다.
4. **보고서 산출** — HTML / PDF 동일 내용 (Jinja2 + WeasyPrint). 모델 비교표, 튜닝 결과, ROC / PR 곡선, feature importance, score 분포, confusion matrix 포함.
5. **주기 스코어링** — 단일 artifact (preprocessor + model + metadata) 파일 1개 + 설정 YAML 1개로 운영 가능. cron 등에서 `auto-ml-score` 호출.

입력 데이터

- 학습 / 평가 데이터를 **별도 Parquet 파일** 로 받는다 (`train_data_path` , `test_data_path`). 내부에서 임의 분할하지 않는다.
- 학습/스코어링에 사용할 컬럼은 **CSV 파일** 로 명시한다. YAML 에서 `features_csv` 에 경로를 지정하면 `load_config` 가 자동으로 읽어 `cfg.features` 를 채운다.

CSV 형식 (필수 컬럼: `name`, `type`, `used`):

```
name,type,used
age,continuous,true
gender,category,true
income,continuous,true
internal_score,continuous,false
```

- `type`
- `continuous` — 연속형 (int / float). 결측·이상치·skew 변환·스케일링 적용.
- `category` — 범주형 (문자열 / 코드). 모델 래퍼에서 자체 인코딩 처리.
- `used`
- `true` 인 행만 학습/스코어링에 사용된다 (`false` /공백 → 제외).
- 운영 중 컬럼을 켜고 끌 때 코드 변경 없이 본 CSV 만 수정하면 된다.

```
features_csv: ./configs/features.csv
```

결측치 처리

전처리 1단계. 학습 데이터에서 컬럼별 채움 값을 계산해 저장하고, 스코어링 시 동일하게 적용한다. 학습/스코어링 일관성을 보장한다.

```
preprocessing:
  numeric_null_strategy: median          # median | mean | constant
  numeric_null_fill_value: 0.0          # constant 일 때만 사용
  categorical_null_strategy: most_frequent # most_frequent | constant
  categorical_null_fill_value: MISSING   # constant 일 때만 사용
```

- 수치형 전략: `median` (이상치에 강건, 기본) / `mean` / `constant`.
- 범주형 전략: `most_frequent` (기본) / `constant`.
- **전부-NaN 가드**: 학습 데이터에서 어떤 수치형 컬럼이 전부 NaN 이면 `median` / `mean` 으로 채움 값을 계산할 수 없어 silent NaN 전파를 막기 위해 명시적 `ValueError` 로 실패한다. `features.csv` 에서 해당 컬럼의 `used` 를 `false` 로 두거나 `constant` 전략으로 전환.
- 별도로 `PreprocessingPipeline.fill_missing_columns(df)` 가 스코어링 시 **컬럼 자체가 통째로 누락된** 케이스에 대비해 학습 시 산출한 기본값 (수치형 `median`, 범주형 최빈값) 으로 채워준다. 부분 NaN 은 imputer 가 처리.

이상치 처리 (백분위수 원저라이징)

전처리 2단계. 학습 데이터의 컬럼별 분위수 (`quantile(lower_q)`, `quantile(upper_q)`) 를 경계로 사용해 원저라이징하고, 스코어링 시 학습 시 저장한 동일 경계를 그대로 적용한다.

```
preprocessing:
  outlier_method: percentile      # percentile | none
  outlier_lower_quantile: 0.01
  outlier_upper_quantile: 0.99
  outlier_action: clip           # clip | null_then_impute
```

- `outlier_method`
- `percentile` — 학습 분포의 (`lower_q`, `upper_q`) 분위수를 경계로 사용. (기본)
- `none` — 비활성.
- `outlier_action`
- `clip` — 경계 밖 값을 경계로 잘라낸다. (기본)
- `null_then_impute` — 경계 밖을 NaN 으로 만든 뒤 학습 시 median 으로 다시 채운다.
- `outlier_lower_quantile < outlier_upper_quantile` 이고 둘 다 `[0, 1]` 범위여야 한다 (위반 시 `ValueError`).

Skew 변환

전처리 2.5단계. 학습 데이터의 컬럼별 skew 를 측정해 임계값 초과인 변수만 자동으로 선택해 분포를 대칭화한다. 학습 시 추정된 변환 파라미터를 저장해 스코어링에 동일 적용한다.

```
preprocessing:
  skew_method: signed_log1p      # signed_log1p | quantile_normal | none
  skew_threshold: 1.0           # |skew| > threshold 인 컬럼만 변환
```

- `skew_method`
- `signed_log1p` — $\text{sign}(x) * \log_{1p}(|x|)$. 음수까지 단조 보존, 무상태. (기본)
- `quantile_normal` — sklearn `QuantileTransformer` 로 학습 분포의 분위수를 표준정규로 매핑. 이상치/skew 모두에 강건. 학습 시 분위수 테이블을 저장.
- `none` — 비활성.
- 변환 대상은 학습 시 한 번만 결정된다. `|skew| > skew_threshold` 인 컬럼만 선택되며, 미선택 컬럼은 스코어링에서도 그대로 통과한다 (분포 일관성).
- 부스팅 트리는 단조 변환에 본질적으로 강건하지만, 선형/신경망 확장이나 분포 가정을 쓰는 진단에는 효과가 있다.

스케일링

전처리 3단계 (마지막). sklearn 의 스케일러를 pandas 입출력으로 감싼 얇은 래퍼.

```
preprocessing:
    scaling_method: standard          # standard | minmax | robust | none
```

- `standard` — `StandardScaler` (평균 0, 분산 1). 기본.
- `minmax` — `MinMaxScaler` ([0, 1]).
- `robust` — `RobustScaler` (median, IQR — 이상치에 강건).
- `none` — 비활성 (passthrough).

부스팅 트리만 쓰면 의미가 작지만, 선형/신경망 확장을 가정해 옵션으로 둔다. 학습 시 fit 된 스케일러는 artifact 에 저장되어 스코어링 시 동일 변환이 적용된다.

변수 선택 (Stability Selection)

전처리 직후, 모델 학습 직전에 적용되는 선택적 단계이다. 단일 fit 한 번의 우연성을 제거하기 위해, 학습 데이터에서 임의의 부분표본을 반복 추출해 **변수가 얼마나 일관되게 선택되는지**(selection probability) 를 측정하고, 임계값 이상으로 자주 선택된 변수만 최종 학습에 사용한다. Meinshausen & Bühlmann (2010) 의 절차를 따른다.

절차

1. **Stratified 부분표본 추출** — 학습 데이터에서 `subsample_ratio` 비율로 `n_subsamples` 번 부분표본을 뽑는다. 클래스 비율이 부분표본마다 유지된다 (sklearn `StratifiedShuffleSplit`).
2. **부분표본별 변수 선택** — `base_estimator` 로 부분표본 한 번당 변수 셋을 뽑는다 (자세한 동작은 아래).
3. **선택 빈도 누적** — 각 변수가 부분표본 몇 번에서 뽑혔는지 세서 빈도 (0.0 ~ 1.0) 를 계산한다.
4. **임계값 이상 채택** — `frequency >= threshold` 인 변수만 최종 채택. 부족하면 `min_selected fallback` 으로 frequency 상위 N개를 보충한다.

Base estimator

```
feature_selection:
  enabled: true
  base_estimator: lasso          # lasso | lgbm
  n_subsamples: 200             # 부분표본 추출 횟수
  subsample_ratio: 0.5          # 각 부분표본 크기 비율 (Meinshausen 권고 = 0.5)
  threshold: 0.6                # 채택 임계값 (보통 0.6~0.8)
  random_state: 42
  min_selected: 1               # threshold 이상이 부족할 때 fallback 상위 N개

# lasso 전용
lasso_C: 0.1                    # L1 규제 강도 (낮을수록 더 sparse)

# lgbm 전용
lgbm_top_k: 30                  # 부분표본당 gain 상위 K개 채택
lgbm_n_estimators: 100          # 부분표본 학습용 LGBM 트리 수 (가볍게)
lgbm_learning_rate: 0.1
```

- **lasso (L1 로지스틱 회귀)** — 부분표본 단위 fit 후 **non-zero coefficient** 변수를 선택. 선형 신호에 강하고 빠르며 해석이 쉽다. 범주형 컬럼은 full-X 기준 **frequency encoding** 으로 1회 사전 변환해 부분표본마다 재인코딩하는 비용·bias 를 피한다.
- **lgbm (LightGBM gain 중요도)** — 부분표본 단위 fit 후 gain 중요도 상위 **lgbm_top_k** 개(중요도 > 0 만) 를 선택. 비선형·상호작용을 포착하지만 부분표본당 fit 비용이 크다. 범주형은 pandas category dtype 으로 LightGBM native 처리.

선택 기준이 다르므로 두 base estimator 가 항상 같은 변수를 뽑지 않는다. 선형 효과는 lasso 가, 상호작용·비선형은 lgbm 이 더 잘 찾는 경향이다.

설정 가이드

- **n_subsamples** — 표준 100~500. 소규모(< 5k 행)면 50~100 도 충분, 큰 데이터·고차원이면 200+. 추정 분산은 $\sqrt{n}$ 으로 줄어든다.
- **subsample_ratio** — 원 논문 권고 0.5. 부분표본이 너무 크면 모든 표본이 비슷해져 선택이 안정화되지 않고, 너무 작으면 fit 자체가 불안정해진다.
- **threshold** — 0.6 보수적, 0.8 매우 보수적. false-positive 통제 수준의 trade-off. 비교적 적은 채택을 원하면 0.7~0.8.
- **min_selected** — threshold 이상 변수가 부족할 때 frequency 상위 N개로 fallback. 모델 입력이 비어 학습이 실패하는 사고를 막는다. 운영 안정성용 안전망이며 평소엔 거의 발동되지 않아야 한다 (발동 시 WARN 로그).
- **lasso_C** — LogisticRegression(C=...) 의 역규제 강도. **낮을수록 규제 강함 → 더 sparse**. 0.1 이 시작점, feature 가 많이 살아 남으면 0.01 로 조이고, 너무 잘려나가면 1.0 으로 푼다.

- `lgbm_top_k` — 부분표본당 채택 상한. 일반적으로 전체 feature 의 30~50%. 너무 크면 거의 모든 feature 가 한 번씩 뿔혀 threshold 이상이 폭증한다.

Fallback / 실패 처리

- 부분표본 한 개 fit 이 실패해도 warning 만 남기고 다음 부분표본으로 진행 (예: 극단적인 stratify 결과로 한 클래스가 비는 경우).
- 모든 부분표본이 실패하면 `RuntimeError . base_estimator` 설정 점검 필요.
- threshold 이상 변수가 `min_selected` 미만이면 frequency 상위 N개로 fallback 하고 `fallback_used=True` 가 메타데이터에 기록된다.

산출물

`SelectionResult` 에 다음이 담긴다:

- `selected_features` — 채택된 변수 목록 (입력 컬럼 순서 유지).
- `frequencies` — 모든 변수의 선택 빈도 (0.0 ~ 1.0).
- `n_subsamples` — 실제 사용된 부분표본 수 (실패 제외).
- `fallback_used` — fallback 발동 여부.

선택 결과는 artifact 메타데이터에 저장되어 **스코어링 시 동일 컬럼 셋으로 강제 적용**된다. HTML/PDF 리포트에는 변수별 selection frequency 막대와 채택/제외 라벨이 표기된다 (예: `examples/credit/` 리포트 6번 섹션 참고).

언제 켜고, 언제 끄나

- **켄다** — feature 수가 많고 일부가 noise 일 가능성이 있는 경우, 운영 안정성과 설명 가능성이 중요한 경우, 학습/스코어링 컬럼 셋을 명시적으로 줄이고 싶은 경우.
- **끈다 (enabled: false)** — 도메인 지식으로 이미 features.csv 가 정제됐고 feature 수가 적은 경우, 부스팅 트리 자체의 내장 selection 으로 충분하다고 판단되는 경우.

구현 메모

코드: `auto_ml/feature_selection/stability.py` 의 `StabilitySelector . fit_select(X, y) -> SelectionResult` 가 핵심 entry. 주요 흐름:

1. **base_estimator 별 사전 인코딩 1회** (`_encode_for_lasso` / `_prepare_for_lgbm`). lasso 는 범주형에 **full-X frequency encoding**, lgbm 은 pandas category dtype 으로 변환. 부분표본마다 재인코딩하지 않아 비용과 bias 를 줄인다.
2. **StratifiedShuffleSplit(n_splits=n_subsamples, train_size=subsample_ratio, random_state=...)** 으로 부분표본 인덱스 시퀀스를 생성. 각 인덱스에 대해 base 선택 함수 호출.
3. **부분표본 선택**: - `_lasso_select` — `LogisticRegression(solver="liblinear", C=lasso_C, penalty="l1", max_iter=200) . abs(coef) > 1e-8` 인 컬럼을 채택. - `_lgbm_select` — `LGBMClassifier(...)` fit, gain importance 상위 `lgbm_top_k` 개 중 `importance > 0` 만 채택.

4. **실패 처리** — 부분표본 단일 실패는 try/except 로 잡아 warning 후 continue. 모든 부분표본이 실패하면 RuntimeError .
5. **threshold + fallback** — `frequencies[f] = count[f] / n_done . frequency ≥ threshold` 인 변수만 채택, 부족하면 빈도 상위 `min_selected` 개로 fallback 하고 `fallback_used=True` 가 메타에 기록된다.

모델 래퍼

`auto_ml/models/` 의 세 래퍼 (`LGBMModel` , `XGBModel` , `CatBoostModel`) 는 모두 동일한 인터페이스(`fit` / `predict_proba` / `feature_importance`) 를 따른다. 차이는 범주형 처리 방식과 `early_stopping` 구현이다.

모델	범주형 처리	early_stopping 적용	비고
<code>LGBMModel</code>	pandas category dtype native	<code>callbacks=[early_stopping(rounds)]</code>	가장 빠름
<code>XGBModel</code>	컬럼별 LabelEncoder (학습 시 fit → 스코어링 재사용, <code>_encoders</code> 보관)	생성자 인자 <code>early_stopping_rounds</code> (XGB 2.x sklearn API)	폐쇄망 호환을 위해 native cat 대신 LabelEncoder 사용
<code>CatBoostModel</code>	native (<code>cat_features</code> 인덱스 전달)	학습 인자 <code>early_stopping_rounds</code>	<code>allow_writing_files:</code> <code>false</code> 권장 — 운영 임시 파일 차단

3 모델 모두 동일한 전처리 결과와 (활성 시) 변수 선택 채택 컬럼 셋을 공유한 채 병렬 학습되고, **테스트 데이터 primary_metric 점수가 가장 좋은 모델 1개**가 best 로 선정되어 artifact 에 저장된다.

Trainer 흐름

`auto_ml/models/trainer.py` 의 `ModelTrainer` 가 모델별로 다음을 수행:

1. (옵션) `HyperparameterOptimizer` 가 Optuna TPE 로 `tuning.cv_folds` OOF 평균 `primary_metric` 을 최대화하는 파라미터 탐색.
2. 튜닝된(또는 `fixed_params` 만) 파라미터로 `training.cv_folds` StratifiedKFold OOF 평가. 각 fold 의 `best_iter` 평균이 리포트의 `best_iter (avg)` 로 표기됨.
3. 학습 데이터 전체로 최종 fit (`early_stopping` 비활성).
4. 테스트 데이터로 평가 → 모델별 holdout 점수 산출.

하이퍼파라미터 최적화

각 모델 블록에 `fixed_params` (항상 적용) 와 `search_space` (Optuna 탐색 범위) 를 둔다. `tuning.enabled: true` 이고 모델별 `search_space` 가 비어있지 않으면 Optuna TPE 가 KFold OOF `primary_metric` 을 최대화하는 파라미터를 찾는다.

설정

```
training:
  cv_folds: 5                # 최종 OOF 평가용 fold 수
  random_state: 42
  early_stopping_rounds: 50  # 부스팅 모델 조기종료 라운드 수
  primary_metric: roc_auc    # 모델 선택 / 튜닝 목적함수

tuning:
  enabled: true
  n_trials: 30               # 모델당 시도 횟수
  timeout: null              # 초 단위 wall-clock 상한 (null 이면 제한 없음)
  cv_folds: 3                # 튜닝 단계 fold 수 (보통 training.cv_folds 보다 작게)
  random_state: 42

models:
  lgbm:
    enabled: true
    fixed_params: { objective: binary, metric: auc, verbose: -1, n_estimators: 2000 }
    search_space:
      learning_rate: { type: float, low: 0.01, high: 0.3, log: true }
      num_leaves:    { type: int,   low: 15,   high: 255 }
      reg_lambda:    { type: float, low: 1.0e-8, high: 10.0, log: true }
```

`search_space` 형식

- **type: float** — `low`, `high`, `log` (선택, 기본 false).
- **type: int** — `low`, `high`, `log` (선택), `step` (선택, 기본 1).
- **type: categorical** — `choices: [...]`.

`search_space` 가 비어있으면 해당 모델은 `fixed_params` 만으로 학습한다 (튜닝 스킵).

primary_metric 지원 목록

auto_ml/reporting/metrics.py:SUPPORTED_METRICS 가 단일 진실 출처: roc_auc | pr_auc | accuracy | precision | recall | f1 | ks | lift . 모두 **최대화** 방향이다. 도메인별 권장:

- roc_auc — 일반 (기본).
- pr_auc — positive 비율이 매우 낮은 불균형 데이터.
- ks — 신용평점 컨텍스트.
- f1 / precision / recall — 임계값 0.5 기준이라 운영 임계값을 다르게 쓰는 경우엔 roc_auc / pr_auc 가 더 안전.

early_stopping

training.early_stopping_rounds 는 부스팅 모델 세 종류에 모두 적용되지만 내부 구현은 다르다 (위 모델 래퍼 표 참고). OOF 평가 단계의 fold 학습에는 적용되고 **최종 fit 에는 적용되지 않는다** (early_stopping 이 검증셋을 요구하므로).

설치

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip install -e .
```

WeasyPrint 는 시스템 폰트와 일부 라이브러리(libpango 등) 가 필요하다. 폐쇄망에서는 wheelhouse + 시스템 패키지를 함께 배포한다.

사용

```
# 1) 더미 데이터 생성 (검증용)
python examples/make_dummy_data.py

# 2) 학습 — 산출물:
#   artifacts/models/best.joblib
#   artifacts/reports/report.html, report.pdf
auto-ml-train --config configs/example.yaml

# 3) 스코어링 — 산출물:
#   artifacts/scores/scores.parquet (id_columns + score + prediction)
auto-ml-score --config configs/example.yaml
```

코드에서 직접 호출하는 예시는 `examples/run_train.py`, `examples/run_score.py` 참고.

타이타닉 end-to-end 예시

실제 공개 데이터셋(Titanic, 891 rows) 으로 전체 파이프라인을 검증할 수 있다.

```
# 데이터 준비 (GitHub 에서 1회 다운로드 → data/titanic/{train,test,score_input}.parquet)
python examples/titanic/prepare_data.py

# 학습 — 산출물: artifacts/titanic/{models,reports,logs}/...
auto-ml-train --config examples/titanic/config.yaml

# 스코어링 — 산출물: artifacts/titanic/scores/scores.parquet
auto-ml-score --config examples/titanic/config.yaml
```

검증 시 약 10초 내외 (Optuna 10 trials × 3 모델 × 3 fold) 에 best 모델 ROC-AUC ≈ 0.86 을 재현한다. 폐쇄망에서는 `TITANIC_CSV` 환경변수에 사전 배포한 CSV 경로를 지정하면 `prepare_data.py` 가 네트워크 없이 동일 산출물을 만든다.

폐쇄망 이관

다음 4가지만 옮기면 된다:

1. 패키지 wheel (`pip wheel -r requirements.txt -w wheelhouse/`)
2. 본 repo 자체 (또는 `pip install -e .` 으로 wheel 생성)
3. 학습 산출물 `artifacts/models/best.joblib`
4. 스코어링 설정 YAML

설정

전체 옵션은 `configs/example.yaml` 참고. 모든 키는 `auto_ml/config.py` 의 dataclass 와 1:1 대응한다.

작업 로그

실행마다 stage(`train` / `score`) 별 별도 로그 파일이 자동 생성된다.

```
artifacts/logs/
├── train_20260425_143052.log
└── score_20260425_180001.log
```

설정 (`configs/example.yaml` 의 `logging` 섹션):

```
logging:
  log_dir: ./artifacts/logs
  level: INFO          # DEBUG | INFO | WARNING | ERROR
  to_stdout: true
  to_file: true
```

콘솔 출력과 파일 출력은 동시에 활성화 가능하며, `to_file: false` 로 두면 파일은 만들지 않는다. 각 로그는 시작/종료 마커, 단계별 진행, 모델 튜닝 결과, 산출물 경로, 총 소요 시간을 포함한다.

운영 메모

- 타깃은 0/1 만 허용 (`utils/validation.py` 가 검증).
- best 선정은 테스트 데이터 점수 기준.
- 학습 시 사용한 features 정의가 artifact 메타데이터에 저장되어 스코어링 시 동일 컬럼 셋·동일 전처리·동일 모델로 처리된다.
- 스코어링 결과 컬럼: `<id_columns> + score + prediction`.
- 학습 데이터에서 어떤 수치형 컬럼이 전부 NaN 이면 결측·이상치 단계가 명시적 `ValueError` 로 실패한다 (silent NaN 전파 방지). `features.csv` 의 `used` 를 `false` 로 두거나 해당 단계를 비활성화/ `constant` 전략으로 전환.